

ClauseIQ

Insurance Policy RAG System

PHASE 1 -- FOUNDATION

Complete Project Scaffold & RAG Pipeline

26

Files Created

1,851

Lines of Code

8

Modules Built

Python 3.13

FastAPI

Streamlit

FAISS

Mistral AI

Sentence Transformers

PyMuPDF

Docker

Generated: August 30, 2026 | Repository: github.com/adityarana2610/clauseiq

Author: Aditya Rana | Branch: main | Commit: e90d8b0

SECTION 01

Project Overview

Goals, architecture, and the eval-first philosophy

1. Project Overview

What is ClauselQ?

ClauselQ is an enterprise-grade Retrieval-Augmented Generation (RAG) system built for insurance policy Q&A. Given a set of insurance policy PDFs, the system allows users to ask natural language questions and receive grounded, cited answers backed by real document excerpts -- not hallucinated responses.

The system is designed to be fully defensible in a technical interview: every design decision has a measurable outcome, every experiment produces a number.

Core Philosophy: Eval-First

The roadmap mandates building the 50-question evaluation set before pipeline code. This forces every design decision to be judged against real metrics -- not guesses. The four interview-ready sentences you will say by the end:

- I tested chunk sizes 500, 800, 1200 tokens -- 800 performed best because [reason]
- Adding the reranker improved Recall@5 from X% to Y% on my eval set
- Hallucination rate dropped from X% to Y% after prompt tuning
- Here is a question my system correctly refused to answer: [example]

Technology Stack

Component	Choice	Rationale
Language	Python 3.13	ML/NLP ecosystem standard
Backend	FastAPI	Async, auto-docs, production-grade
UI	Streamlit	Rapid prototyping, no frontend overhead
PDF Parsing	PyMuPDF 1.24.11	Fastest Python PDF library
Embeddings	Sentence Transformers	Local, all-MiniLM-L6-v2 (384-dim)
Vector DB	FAISS (IndexFlatIP)	Zero infra, cosine similarity
Reranker	ms-marco-MiniLM	Two-stage: recall + precision
LLM	Mistral (mistral-small)	Cost-efficient, OpenAI-compatible
Containers	Docker + Compose	Reproducible deployment

Why Not Just Paste the PDF Into GPT?

- Context window limits: large PDFs exceed any model context window
- Cost: sending full documents on every query is expensive at scale
- No retrieval control: cannot filter by document type or metadata
- No citations: users cannot verify which document the answer came from
- Hallucination rate: without grounding, models invent policy details
- RAG gives: precision retrieval + grounded generation + measurable quality

SECTION 02

Project Structure

Full file tree and module descriptions

2. Project Structure

All 26 files are committed to github.com/adityarana2610/clauseiq on branch main (commit e90d8b0). Local path: e:\RAG\clauseiq\

```
PLAINTEXT
clauseiq/
|-- app/
|   |-- ingestion/
|   |   |-- pdf_parser.py      # PyMuPDF extraction -- text + metadata per page
|   |   |-- cleaner.py        # Remove headers/footers, normalize whitespace
|   |   +-- chunker.py         # Token-based chunker (configurable size + overlap)
|   |-- retrieval/
|   |   |-- embedder.py        # Sentence Transformers (all-MiniLM-L6-v2)
|   |   |-- vector_store.py    # FAISS + metadata filtering + save/load
|   |   +-- reranker.py        # Cross-encoder (ms-marco-MiniLM-L-6-v2)
|   |-- generation/
|   |   |-- prompt.py          # System prompt + hallucination guardrail
|   |   +-- llm.py             # Mistral API wrapper
|   |-- evaluation/
|   |   |-- metrics.py         # Recall@k, Hit Rate, Refusal Rate
|   |   +-- eval_runner.py     # CLI runner -> auto-fills experiments/results.md
|   |-- api/
|   |   +-- main.py            # FastAPI: /health /documents /upload /ask
|   +-- pipeline.py            # End-to-end RAG orchestrator
|-- ui/
|   +-- streamlit_app.py       # Upload + Q&A + source citation UI
|-- data/
|   +-- eval_set_template.json # 50-Q schema (13 pre-seeded)
|-- experiments/
|   +-- results.md             # Chunking experiment table
|-- docker/
|   |-- Dockerfile
|   +-- docker-compose.yml
|-- requirements.txt
|-- .env.example
+-- .gitignore
```

SECTION 03

Ingestion Pipeline

PDF parsing, cleaning, and token-based chunking

3. Ingestion Pipeline

3.1 PDF Parser -- app/ingestion/pdf_parser.py

Uses PyMuPDF (fitz) to extract text and metadata from each page of every PDF in the source directory. Each page becomes a structured dict preserving all metadata needed for citations.

Document Type Auto-Detection

doc_type is inferred from the filename prefix: policy_* -> policy, endorsement_* -> endorsement, claim_* -> claim, exclusion_* -> exclusion. This enables metadata filtering at retrieval time -- a key interview talking point.

PYTHON

```
# Page dict structure returned by pdf_parser
{
  "text": "The annual deductible for flood...",
  "page": 4,          # 1-indexed
  "document": "policy_home_standard_001",
  "document_path": "E:/RAG/clauseiq/data/raw_pdfs/policy_home_standard_001.pdf",
  "doc_type": "policy"
}
```

3.2 Text Cleaner -- app/ingestion/cleaner.py

Removes PDF extraction noise before chunking. Noise degrades embedding quality.

- Removes: 'Page X of Y', standalone page numbers, 'Confidential', divider lines
- Collapses 3+ consecutive blank lines into a single blank line
- Normalizes multiple spaces/tabs to single space
- Filters pages under 50 characters after cleaning (scanned/blank pages)

3.3 Chunker -- app/ingestion/chunker.py

Splits cleaned text into overlapping token-based chunks using tiktoken (cl100k_base -- same as GPT-4/Mistral). Token-based chunking gives deterministic, model-aligned sizes unlike character-based splitting.

Why Token-Based Chunking?

LLMs have token limits, not character limits. A character-based splitter at 3,200 chars might produce chunks of 900 tokens (too large) or 400 tokens (too small). Token-based chunking gives exact control over the model context budget.

Parameter	Default	Week 2 Experiment Values
chunk_size	800 tokens	500 / 800 / 1200
chunk_overlap	100 tokens	50 / 100 / 150

SECTION 04

Retrieval Layer

Embeddings, FAISS vector store, cross-encoder reranker

4. Retrieval Layer

4.1 Embedder -- `app/retrieval/embedder.py`

- Model: all-MiniLM-L6-v2 -- 22M params, runs on CPU, 384-dimensional output
- `normalize_embeddings=True` -- enables cosine similarity via dot product with FAISS IndexFlatIP
- Batch size 64 for chunk embedding; single call for query embedding

4.2 Vector Store -- `app/retrieval/vector_store.py`

FAISS IndexFlatIP with a parallel Python list for chunk metadata. FAISS stores only float32 vectors; text and metadata live in the Python list.

Interview Point -- IndexFlatIP vs IndexIVFFlat

IndexFlatIP is exact nearest-neighbor: $O(n)$ time, 100% recall. Fine for a portfolio project with ~10K chunks. In production at 10M+ vectors I would use IndexIVFFlat (Inverted File Index) for approximate nearest-neighbor at $O(\log n)$ -- trading ~1% recall for 100x speed improvement.

- Metadata filtering: pre-filter by `doc_type` or document name before returning top-K
- Persistence: FAISS index as `.faiss` binary + metadata as `.meta` pickle
- Over-retrieval: fetches $K*5$ candidates when filters active, then post-filters to K

4.3 Reranker -- `app/retrieval/reranker.py`

Two-stage retrieval: bi-encoder for recall (FAISS), cross-encoder for precision.

Stage	Model Type	Speed	Accuracy	Purpose
Stage 1 -- FAISS	Bi-encoder	Fast (ms)	Good recall	Top-20 candidates
Stage 2 -- Reranker	Cross-encoder	Slower	High precision	Top-20 -> Top-5

Interview Point -- Why Two Stages?

A bi-encoder encodes query and document separately then compares vectors -- fast but loses cross-attention signal. A cross-encoder reads query+document together with full attention -- much more accurate but $O(n)$ model calls. Two-stage = FAISS for recall + cross-encoder for precision. The Week 2 eval will measure the Recall@5 delta.

SECTION 05

Generation Layer

Prompts, Mistral API, hallucination control

5. Generation Layer

5.1 Prompt Templates -- app/generation/prompt.py

System prompt encodes the hallucination guardrail as explicit numbered rules:

```
PYTHON
SYSTEM_PROMPT = """
You are ClauseIQ, an expert assistant for insurance policy documents.

Rules you MUST follow:
1. Only use information from the provided context sections below.
2. If the answer is not in the context, say:
    "I cannot find this information in the provided documents."
3. Do NOT hallucinate or use general knowledge not present in context.
4. Always cite your source: document name and page number per fact.
5. Be concise but complete.
"""
```

- Rule 1 -- Strict grounding: blocks parametric knowledge use
- Rule 2 -- Explicit refusal phrase: measurable in eval (Refusal Rate metric)
- Rule 4 -- Citation requirement: every fact must have document + page source
- Temperature 0.1 -- near-deterministic output for factual insurance Q&A

5.2 Mistral LLM Wrapper -- app/generation/llm.py

Model	Use Case	Approx Cost
open-mistral-7b	Development / high-volume eval runs	~\$0.07 / 1M tokens
mistral-small-latest	Demo + final eval (default)	~\$0.20 / 1M tokens
mistral-large-latest	LLM-as-judge eval scoring	~\$2.00 / 1M tokens

SECTION 06

RAG Pipeline

Central orchestrator connecting all components

6. RAG Pipeline -- app/pipeline.py

RAGPipeline is the single class that coordinates ingestion, retrieval, reranking, and generation. This is the class you will demonstrate in interviews.

Pipeline Flow

```
PLAINTEXT
PDF Files
| pdf_parser.extract_all_pdfs()    -> text + metadata per page
| cleaner.clean_pages()           -> remove noise
| chunker.chunk_pages(size=800)   -> token-based chunks
| embedder.embed(texts)           -> 384-dim float32 vectors
| vector_store.add_chunks()        -> FAISS index built
| vector_store.save()              -> persist to disk

User Query
| embedder.embed_query(q)         -> single query vector
| vector_store.search(top_k=20)    -> FAISS cosine similarity
| reranker.rerank(top_k=5)         -> cross-encoder re-scoring
| llm.generate(question, chunks)   -> Mistral API call
| {answer, sources, model}         -> grounded response
```

Usage

```
PYTHON
# Ingest
pipeline = RAGPipeline(chunk_size=800, use_reranker=True)
pipeline.ingest('data/raw_pdfs')
pipeline.save('data/faiss_index')

# Query
pipeline = RAGPipeline.load('data/faiss_index')
result = pipeline.ask('What is the flood damage deductible?')
# result['answer'] -> grounded answer with citations
# result['sources'] -> [{document, page, doc_type, score}]

# Retrieval only (no LLM) -- for Week 2 eval experiments
chunks = pipeline.retrieve(query, filter_doc_type='policy')
```

SECTION 07

Evaluation Framework

Metrics, eval set schema, experiment table

7. Evaluation Framework

7.1 Metrics -- app/evaluation/metrics.py

Metric	Definition	Phase
Recall@k	Expected doc+page found in top-k retrieved chunks?	Week 2
Hit Rate	Fraction of answerable Qs with >=1 relevant chunk in top-k	Week 2
Refusal Rate	Fraction of unanswerable Qs where model correctly refused	Week 3

7.2 Eval Set Schema -- data/eval_set_template.json

```
JSON
{
  "id": "Q001",
  "question": "What is the annual deductible for flood damage?",
  "expected_answer": "The annual deductible is $2,500.",
  "expected_document": "policy_home_standard_001",
  "expected_page": 4,
  "doc_type": "policy",
  "category": "deductibles",
  "is_answerable": true
}

// Unanswerable question example:
{
  "id": "UNANSWERABLE_001",
  "question": "What is the CEO name of the insurance company?",
  "expected_answer": null,
  "is_answerable": false,
  "reason": "CEO names are not in insurance policy documents"
}
```

Why Include Unanswerable Questions?

10 of the 50 questions are deliberately unanswerable. The system must refuse them using the exact phrase from the system prompt. Refusal Rate = correct refusals / total unanswerable questions. This is your hallucination control metric.

7.3 Experiment Table -- experiments/results.md

Auto-populated by the eval runner. Fill these in during Week 2:

Config	Chunk Size	Reranker	Recall@5	Hit Rate
baseline_500	500	No	-- run eval --	-- run eval --
baseline_800	800	No	-- run eval --	-- run eval --
baseline_1200	1200	No	-- run eval --	-- run eval --
best_no_rerank	800	No	-- run eval --	-- run eval --
best_with_rerank	800	Yes	-- run eval --	-- run eval --

Run an experiment:

```
BASH
python -m app.evaluation.eval_runner \
  --index_path data/faiss_index \
  --eval_set data/eval_set.json \
  --k 5 --run_llm --label best_with_rerank
```

SECTION 08

API & User Interface

FastAPI backend and Streamlit Q&A interface

8. API & User Interface

8.1 FastAPI Backend -- app/api/main.py

Endpoint	Method	Description
/health	GET	Status, document count, chunk count
/documents	GET	List indexed docs with page and chunk counts
/upload	POST	Ingest a PDF (multipart/form-data), persists index
/ask	POST	Answer question with optional doc_type/document filter

```
BASH
# Start API server
uvicorn app.api.main:app --reload --port 8000

# Auto-docs available at:
http://localhost:8000/docs
```

8.2 Streamlit UI -- ui/streamlit_app.py

- Sidebar: PDF upload -> POST /upload -> shows ingestion summary
- Sidebar: Document type filter (All / policy / endorsement / claim / exclusion)
- Sidebar: Live health check with doc count and chunk count
- Main: Question input -> POST /ask -> answer + source cards
- Sources: blue-accented cards with document name, page number, doc_type, score

```
BASH
streamlit run ui/streamlit_app.py
```

9. Docker

docker/Dockerfile

```
DOCKERFILE
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y libmupdf-dev
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker/docker-compose.yml

Runs FastAPI (port 8000) + Streamlit (port 8501) with shared FAISS index volume:

```
YAML
services:
  api: # FastAPI on :8000, reads MISTRAL_API_KEY from .env
  ui: # Streamlit on :8501, API_URL=http://api:8000
volumes:
  clauseiq_data: # Persists data/faiss_index between restarts

# Run everything:
docker-compose -f docker/docker-compose.yml up --build
```

SECTION 09

Interview Defense

Questions you will be asked and answers to prepare

10. Interview Defense Guide

The Four Non-Negotiable Sentences

- I tested chunk sizes 500, 800, and 1,200 tokens -- 800 performed best because [from results.md]
- Adding the cross-encoder reranker improved Recall@5 from X% to Y% on my 50-question eval set
- Hallucination rate was X% before prompt tuning. After explicit refusal rules it dropped to Y%
- Here is a question my system correctly refused: [example from eval set]

Q&A Preparation

Q: Why not just paste the PDF into GPT?

A: Context window limits, cost at scale, no retrieval control, no citations, and uncontrolled hallucination rate. RAG gives measurable, grounded, citable answers.

Q: Why FAISS over Pinecone or Qdrant?

A: FAISS has zero infrastructure overhead for a portfolio project and scales to millions of vectors on one machine. In production I would use Qdrant for horizontal scaling, access control, and native metadata filtering at the database level.

Q: Why a two-stage retriever?

A: Cross-encoders need one model call per candidate -- $O(n)$ with corpus size. Using FAISS for stage 1 narrows from thousands to 20 candidates; the cross-encoder re-scores just those 20. This gives FAISS recall + cross-encoder precision at low latency cost.

Q: How do you handle out-of-scope questions?

A: The system prompt explicitly instructs the model to say the refusal phrase if the answer is not in the retrieved context. We validated this by running 10 deliberately unanswerable questions and measuring the refusal rate as a metric.

Q: What would you add in production?

A: Hybrid BM25 + vector search for keyword recall on domain jargon, query rewriting for multi-turn conversation, document-level access control, OCR for scanned PDFs, and Qdrant for scalable production vector retrieval.

11. Phase 2 -- What Comes Next

Week 2 Schedule

Day	Task	Output
Day 8	Drop PDFs into data/raw_pdfs/, run ingest	FAISS index built
Day 8	Build 50-question eval set (eval_set.json)	Ground truth established
Day 8	Retrieval-only eval: Recall@5, Hit Rate	Baseline numbers
Day 9-10	Chunk experiment: 500 vs 800 vs 1200 tokens	Best chunk size found
Day 11	LLM generation + unanswerable question test	Hallucination baseline
Day 12	Add page citations to all answers	Citable responses
Day 13-14	Add reranker, re-run eval, compare Recall@5	Reranker delta measured

Your Immediate Action Items

- Copy .env.example -> .env and add your Mistral API key (get free at console.mistral.ai)
- Install dependencies: pip install -r requirements.txt
- Name your PDFs with policy_/endorsement_/exclusion_ prefix and drop in data/raw_pdfs/
- Build the 50-question eval set in data/eval_set.json using the template schema
- Run ingestion and your first retrieval eval

Critical Reminder

Do not skip building the eval set before running experiments. Every metric in your interview story comes from this set. Build it first, run experiments second.